

Assignment-11.5

Name : K.Tejaswini

Ht.no: 2303A51425

Batch.No:21

TaskDescription#1–StackImplementation Prompt:

Create a Python Stack class using a list. Include methods: push, pop, peek and is_empty. Add proper docstrings and it should handle edge cases and errors. Take input from user to demonstrate the functionality of the Stack class.

Code:

```
class Stack:
    """
    A Stack class implemented using a list to demonstrate stack operations: push, pop, peek, and is_empty.
    """
    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def push(self, item):
        """
        Add an item to the top of the stack.

        Parameters:
        item: The item to be added to the stack.
        """
        self.stack.append(item)

    def pop(self):
        """
        Remove and return the item at the top of the stack.

        Returns:
        The item at the top of the stack.

        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Stack is empty. Cannot pop from an empty stack.")
```

```
print("5. Exit")
choice = input("Enter your choice (1-5): ")

if choice == '1':
    item = input("Enter the item to push onto the stack: ")
    stack.push(item)
    print(f'{item}' has been pushed onto the stack.")
elif choice == '2':
    try:
        popped_item = stack.pop()
        print(f'{popped_item}' has been popped from the stack.")
    except IndexError as e:
        print(e)
elif choice == '3':
    try:
        top_item = stack.peek()
        print(f"The item at the top of the stack is: '{top_item}'")
    except IndexError as e:
        print(e)
elif choice == '4':
    if stack.is_empty():
        print("The stack is empty.")
    else:
        print("The stack is not empty.")
elif choice == '5':
    print("Exiting the program.")
    break
else:
    print("Invalid choice. Please enter a number between 1 and 5.")
```

Output:

```

(base) PS C:\Users\WINDOWS\OneDrive\Desktop\Teja 3_2\AI Assit coding> & C:\Users
Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 1
Enter the item to push onto the stack: 3 5 6 7 8
'3 5 6 7 8' has been pushed onto the stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 2
'3 5 6 7 8' has been popped from the stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 3
Stack is empty. Cannot peek from an empty stack.

```

Observation:

The Stack class works correctly by following the Last In, First Out (LIFO) principle, where the most recently pushed element is removed first. The push, pop, peek, and is_empty methods perform as expected, including proper handling of edge cases like popping or peeking from an empty stack. The interactive user input successfully demonstrates the functionality and error handling of the stack implementation.

Task Description #2 – Queue Implementation

Prompt:

Create a Python Queue class using a list. Implement enqueue, dequeue, peek, and size methods. Follow FIFO principle. Add proper docstrings and handle empty queue errors. Take input from user to demonstrate the functionality of the Queue class.

Code:

```

# Create a Python Queue class using a list. Implement enqueue, dequeue, peek, and size methods. Follow FIFO principle. Add proper docstrings and handle errors.
class Queue:
    """
    A simple implementation of a Queue using a list.
    """
    def __init__(self):
        """Initialize an empty queue."""
        self.items = []
    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)
    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)
    def peek(self):
        """Return the item at the front of the queue without removing it. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty queue")
        return self.items[0]
    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)
    def is_empty(self):
        """Return True if the queue is empty, False otherwise."""

```

```

print("\nQueue Operations:")
print("1. Enqueue")
print("2. Dequeue")
print("3. Peek")
print("4. Size")
print("5. Exit")
choice = input("Enter your choice: ")
if choice == '1':
    item = input("Enter the item to enqueue: ")
    queue.enqueue(item)
    print(f"Enqueued: {item}")
elif choice == '2':
    try:
        item = queue.dequeue()
        print(f"Dequeued: {item}")
    except IndexError as e:
        print(e)
elif choice == '3':
    try:
        item = queue.peek()
        print(f"Front of the queue: {item}")
    except IndexError as e:
        print(e)
elif choice == '4':
    print(f"Size of the queue: {queue.size()}")
elif choice == '5':
    print("Exiting...")
    break
else:
    print("Invalid choice. Please try again.")

```

Output:

```
Queue Operations:
1. Enqueue
2. Dequeue
3. Peek
4. Size
5. Exit
Enter your choice: 1
Enter the item to enqueue: 77
Enqueued: 77

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek
4. Size
5. Exit
Enter your choice: 2
Dequeued: 77
```

Observation:

The Queue class correctly follows the First In, First Out (FIFO) principle, ensuring that elements are removed in the same order they were added. All methods including enqueue, dequeue, peek, and size function properly and handle edge cases like empty queue operations. The implementation effectively demonstrates queue behavior using Python lists.

Task Description #3—LinkedList**Prompt:**

Create a Python implementation of a Singly Linked List. Define a Node class and a LinkedList class.

Include methods: insert(data) to add at the end and display() to print all elements. Add proper docstrings and basic error handling. Take input from user to demonstrate the functionality of the LinkedList class.

Code:

```

class LinkedList:
    def display(self):
        """
        Displays the contents of the linked list in a readable format.
        """
        current_node = self.head
        while current_node:
            print(current_node.data, end=' -> ')
            current_node = current_node.next
        print('None') # Indicate the end of the list

    def clear(self):
        """Clears the linked list by setting the head to None, effectively removing all nodes from the list.
        """
        self.head = None

# Example usage:
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Linked List after inserting 10, 20, 30:")
    linked_list.display()

    linked_list.clear()
    print("Linked List after clearing:")
    linked_list.display()

```

Output:

```
ive/Desktop/Teja 3_2/AI Assit coding/lab.py"
Linked List after inserting 10, 20, 30:
10 -> 20 -> 30 -> None
Linked List after clearing:
ive/Desktop/Teja 3_2/AI Assit coding/lab.py"
Linked List after inserting 10, 20, 30:
10 -> 20 -> 30 -> None
Linked List after clearing:
10 -> 20 -> 30 -> None
Linked List after clearing:
None
```

Observation:

The Singly Linked List correctly stores elements in sequential order using node connections. The insert method successfully adds elements at the end of the list, and the display method prints all nodes clearly. The implementation also handles the empty list case properly without errors.

Task Description #4 – Hash Table**Prompt:**

Create a Python HashTable class. Implement insert, search, and delete methods. Add proper docstrings and basic error handling. Take input from user to demonstrate the functionality of the HashTable class.

Code:

```
class HashTable:
    def __init__(self):
        """Initialize the hash table with a fixed size."""
        self.size = 10
        self.table = [None] * self.size

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value) # Update existing key
                    return
            self.table[index].append((key, value)) # Add new key-value pair

    def search(self, key):
        """Search for a value by its key in the hash table."""
        index = self._hash(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
```

```

def delete(self, key):
    """Delete a key-value pair from the hash table."""
    index = self._hash(key)
    if self.table[index] is not None:
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i] # Remove the key-value pair
                return True
        return False # Key not found
# Demonstrate the functionality of the HashTable class
if __name__ == "__main__":
    hash_table = HashTable()

    # Insert key-value pairs
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    # Search for values
    print(hash_table.search("name")) # Output: Alice
    print(hash_table.search("age")) # Output: 30
    print(hash_table.search("city")) # Output: New York
    print(hash_table.search("country")) # Output: None (not found)

    # Delete a key-value pair
    print(hash_table.delete("age")) # Output: True (deleted)
    print(hash_table.search("age")) # Output: None (not found after deletion)

```

Output:

```
ive/Desktop/Teja 3_2/AI Assit coding/lab.py"
Alice
30
New York
None
True
None
```

Observation:

The hash table correctly stores and retrieves key-value pairs using a hash function and chaining for collision handling. The insert, search, and delete operations work efficiently even when multiple keys map to the same index. Edge cases such as deleting or searching for non-existing keys are handled properly without crashing the program.

Task Description #5 – Graph Representation Prompt:

Create a Graph class using an adjacency list (dictionary).

Include methods: `add_vertex`, `add_edge`, and `display`. Add proper docstrings and basic error handling. Take input from user to demonstrate the functionality of the Graph class.

Code:

```
def main():
    if choice == '1':
        vertex = input("Enter vertex to add: ")
        try:
            graph.add_vertex(vertex)
            print(f"Vertex '{vertex}' added successfully.")
        except ValueError as e:
            print(e)

    elif choice == '2':
        vertex1 = input("Enter the first vertex: ")
        vertex2 = input("Enter the second vertex: ")
        try:
            graph.add_edge(vertex1, vertex2)
            print(f"Edge between '{vertex1}' and '{vertex2}' added successfully.")
        except ValueError as e:
            print(e)

    elif choice == '3':
        graph.display()

    elif choice == '4':
        print("Exiting the program.")
        break

    else:
        print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Output:

```
1. Add Vertex
2. Add Edge
3. Display Graph
4. Exit
Enter your choice: 2
Enter the first vertex: a
Enter the second vertex: b
Both vertices must exist in the graph.
```

Menu:

```
1. Add Vertex
2. Add Edge
3. Display Graph
4. Exit
Enter your choice: 3
a b: []
```

Menu:

```
1. Add Vertex
2. Add Edge
3. Display Graph
4. Exit
Enter your choice: 4
Exiting the program.
```

```
(base) PS C:\Users\WINDOWS\OneDrive\Desktop\Teja 3_2\AI Assit coding>
```

Observation:

The graph implementation correctly stores vertices and edges using an adjacency list structure. The `add_vertex` and `add_edge` methods properly update connections between nodes in an undirected manner. The display method successfully shows all vertices along with their connected neighbors, confirming correct functionality.

Task Description #6: Smart Hospital Management System – Data Structure Selection

Prompt:

Create a Python program for Smart Hospital Management System. Implement Emergency Case Handling using a Priority Queue. Patients with higher priority (critical level) should be treated first. Include functions to add patient, treat patient, and display waiting list. Add proper docstrings, and basic error handling. Take input from user to demonstrate the functionality of the Smart Hospital Management System.

Code:

```

py > main
class SmartHospitalManagementSystem:
    def treat_patient(self):
        """Treat the patient with the highest priority (lowest number)."""
        if not self.waiting_list:
            print("No patients in the waiting list.")
            return
        patient = heapq.heappop(self.waiting_list)
        print(f"Treating patient '{patient.name}' with priority {patient.priority}.")

    def display_waiting_list(self):
        """Display the current waiting list of patients."""
        if not self.waiting_list:
            print("No patients in the waiting list.")
            return
        print("Current Waiting List:")
        for patient in sorted(self.waiting_list):
            print(f"Patient: {patient.name}, Priority: {patient.priority}")

def main():
    system = SmartHospitalManagementSystem()
    while True:
        print("\nSmart Hospital Management System")
        print("1. Add Patient")
        print("2. Treat Patient")
        print("3. Display Waiting List")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            name = input("Enter patient name: ")
            try:
                priority = int(input("Enter patient priority (lower number = higher priority): "))
                system.add_patient(name, priority)
            except ValueError:
                print("Error: Priority must be a non-negative integer.")
        elif choice == '2':
            system.treat_patient()
        elif choice == '3':
            system.display_waiting_list()
        elif choice == '4':
            print("Exiting the system. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

Output:

```
ive/Desktop/Teja 3_2/AI Assit coding/lab.py"
```

```
2. Treat Patient
3. Display Waiting List
4. Exit
Enter your choice: 2
Treating patient 'Ramu' with priority 1.
```

```
Smart Hospital Management System
```

```
1. Add Patient
2. Treat Patient
3. Display Waiting List
4. Exit
Enter your choice: 3
No patients in the waiting list.
```

```
Smart Hospital Management System
```

```
1. Add Patient
2. Treat Patient
3. Display Waiting List
4. Exit
Enter your choice: 4
Exiting the system. Goodbye!
```

Observation:

The Priority Queue ensures that patients are treated based on the severity of their condition rather than their arrival time. Patients with critical conditions are given higher priority and attended to first, which supports effective emergency management. The system also properly handles situations when no patients are waiting and maintains efficient performance during patient insertion and treatment.

Task Description #7: Smart City Traffic Control System Prompt:

Create a Smart Traffic Emergency Vehicle System using Priority Queue. Vehicles have name and priority (1 = highest). Implement `add_vehicle()`, `serve_vehicle()`, and `display_queue()`. Include docstrings and basic error handling. Take input from user to demonstrate the functionality of the Smart Traffic Emergency Vehicle System.

Code:

```

class TrafficSystem:

    def serve_vehicle(self):
        """Serve the highest priority vehicle."""
        if not self.queue:
            print("No vehicles in the queue.")
            return
        vehicle = heapq.heappop(self.queue)
        print(f"Serving {vehicle.name} with priority {vehicle.priority}.")

    def display_queue(self):
        """Display the current queue of vehicles."""
        if not self.queue:
            print("The queue is empty.")
            return
        print("Current Queue:")
        for vehicle in sorted(self.queue):
            print(f"{vehicle.name} (Priority: {vehicle.priority})")

def main():
    system = TrafficSystem()
    while True:
        print("\nMenu:")
        print("1. Add Vehicle")
        print("2. Serve Vehicle")
        print("3. Display Queue")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            name = input("Enter vehicle name: ")
            try:
                priority = int(input("Enter vehicle priority (1 = highest): "))
                system.add_vehicle(name, priority)
            except ValueError:
                print("Invalid input for priority. Please enter a number.")
        elif choice == '2':
            system.serve_vehicle()
        elif choice == '3':
            system.display_queue()
        elif choice == '4':
            print("Exiting the system.")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

Output:

```
1. Add Vehicle
2. Serve Vehicle
3. Display Queue
4. Exit
Enter your choice: 1
Enter vehicle name: bike
Enter vehicle priority (1 = highest): 2
Added bike with priority 2 to the queue.
```

```
Menu:
1. Add Vehicle
2. Serve Vehicle
3. Display Queue
4. Exit
Enter your choice: 2
Serving Ambulance with priority 1.
```

```
Menu:
1. Add Vehicle
1. Add Vehicle
2. Serve Vehicle
3. Display Queue
4. Exit
Enter your choice: 3
Current Queue:
bike (Priority: 2)
car (Priority: 3)
```

```
Menu:
1. Add Vehicle
2. Serve Vehicle
3. Display Queue
4. Exit
Enter your choice: 4
Exiting the system.
(base) PS C:\Users\WINDOWS\OneDrive\Desktop\Teja 3_2\AI Assit coding>
```

Observation:

The Priority Queue ensures that emergency vehicles such as ambulances and fire trucks are reserved before normal vehicles regardless of arrival order. This structure was chosen because traffic management requires priority-based handling rather than simple FIFO processing. The implementation successfully demonstrates efficient insertion and removal based on priority levels.

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge Prompt:

Create a Python program implementing an Order Processing System using a Queue. Include enqueue (add order), dequeue (process order), and display methods. Add proper docstrings, and basic error handling. Take input from user to demonstrate the functionality of the Order Processing System.

Code:

```

class OrderProcessingSystem:
    def dequeue(self):
        """
        Process the next order in the queue.
        Returns:
        str: The processed order, or a message if the queue is empty.
        """
        if not self.orders:
            return "No orders to process."
        order = self.orders.pop(0)
        return f"Order '{order}' has been processed."
    def display(self):
        """
        Display all orders currently in the queue.
        Returns:
        str: A list of current orders or a message if the queue is empty.
        """
        if not self.orders:
            return "No orders in the queue."
        return "Current orders in the queue:\n" + "\n".join(self.orders)
def main():
    """Main method to run the order processing system with a console menu."""
    system = OrderProcessingSystem()
    while True:
        print("\nOrder Processing System Menu:")
        print("1. Add Order")
        print("2. Process Order")
        print("3. Display Orders")
        print("4. Exit")
        choice = input("Enter your choice (1-4): ")
        if choice == '1':
            order = input("Enter the order to add: ")
            system.enqueue(order)
        elif choice == '2':
            result = system.dequeue()
            print(result)
        elif choice == '3':
            result = system.display()
            print(result)
        elif choice == '4':
            print("Exiting the Order Processing System. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 4.")

```

Output:

```
rs/WINDOWS/OneDrive/Desktop/Teja 3_2/AI Assit coding/lab.py"
```

```
1. Add Order
2. Process Order
3. Display Orders
4. Exit
Enter your choice (1-4): 1
Enter the order to add: chacos
Order 'chacos' added to the queue.
```

```
Order Processing System Menu:
1. Add Order
2. Process Order
3. Display Orders
4. Exit
Enter your choice (1-4): 2
Order 'sandwich' has been processed.
```

```
Order Processing System Menu:
1. Add Order
2. Process Order
3. Display Orders
4. Exit
Enter your choice (1-4): 3
Current orders in the queue:
Ice cream
chacos
```

Observation:

The Order Processing System correctly follows the FIFO principle, ensuring fairness in handling customer orders. The Queue data structure was chosen because it processes elements in the exact order they are inserted. This makes it the most suitable and logical structure for managing sequential order execution in an e-commerce system.